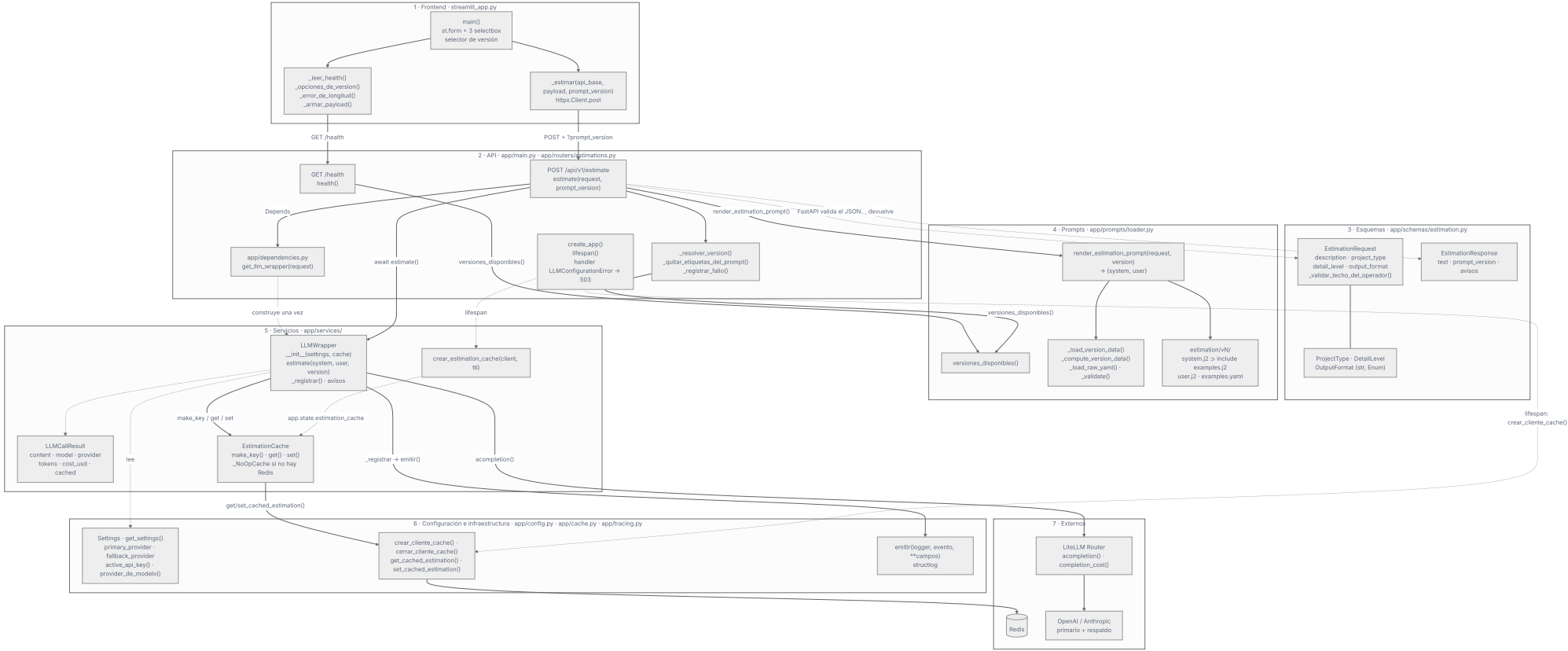


Arquitectura de lidr_4

Estimador de proyectos con FastAPI, LiteLLM, prompts Jinja2 versionados y frontend Streamlit · rama entrega/ldr_4 · 5 de octubre de 2026

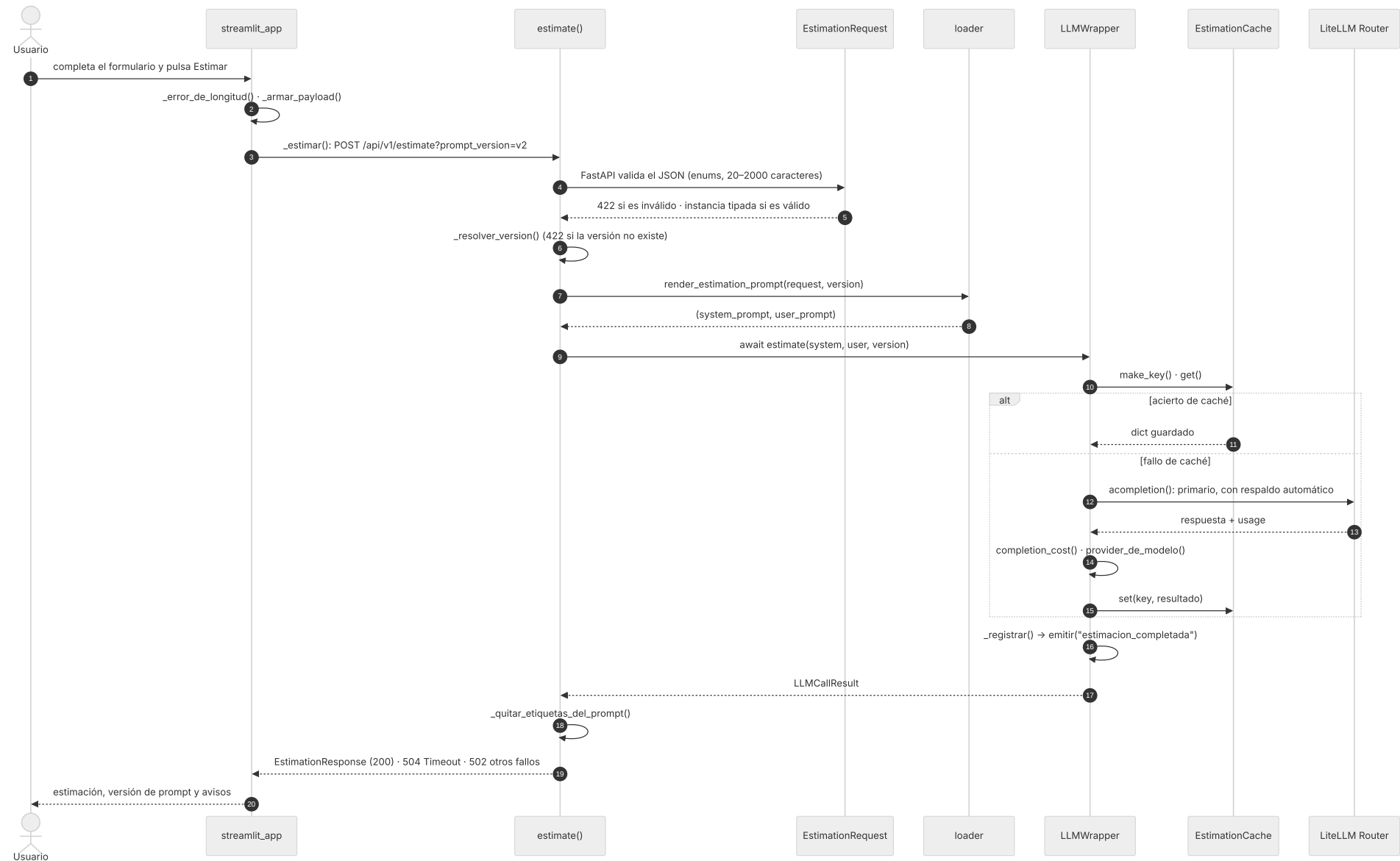
1· Capas y llamadas

Cada caja es un módulo o una clase con sus funciones y métodos; las flechas llevan el nombre de la llamada. Línea continua: llamada durante un request. Línea punteada: arranque, construcción o uso de un tipo.



2 · Secuencia de una estimación

Recorrido de un envío del formulario, con acierto y fallo de caché. El 422 corta antes de renderizar el prompt: una entrada inválida no gasta tokens.



3 · Del formulario al objeto tipado: un ejemplo

Elección concreta del usuario en el formulario de Streamlit, seguida por el código real hasta el prompt.

Tipo de proyecto: **SaaS web** → [web_saas](#)

Nivel de detalle: **Detallado** → [detailed](#)

Formato de salida: **Tabla de fases** → [phases_table](#)

Versión del prompt: [v2](#)

Paso 1 · El formulario captura las claves, no las etiquetas (streamlit_app.py)

Cada selectbox recibe como opciones las claves del enum y usa `format_func` para mostrar la etiqueta en castellano. El valor que devuelve es la clave.

El usuario ve «SaaS web», «Detallado» y «Tabla de fases»; el código recibe `web_saas`, `detailed` y `phases_table`.

```
PROJECT_TYPES = {
    "mobile_app": "App móvil",
    "web_saas": "SaaS web",          # ← elegido
    "internal_tool": "Herramienta interna",
    "data_pipeline": "Pipeline de datos",
}

DETAIL_LEVELS = {"summary": "Resumen", "medium": "Medio", "detailed": "Detallado"}
OUTPUT_FORMATS = {"phases_table": "Tabla de fases", "line_items": "Partidas detalladas",
                  "narrative": "Narrativa"}

with st.form("estimacion", clear_on_submit=False):
    description = st.text_area("Descripción del proyecto", height=180, ...)
    c1, c2, c3 = st.columns(3)
    with c1:
        project_type = st.selectbox(
            "Tipo de proyecto",
            options=list(PROJECT_TYPES),      # valores: "mobile_app", "web_saas", ...
            format_func=PROJECT_TYPES.get,    # lo que ve el usuario: "SaaS web"
            key="project_type_entrada",
        )
    with c2:
        detail_level = st.selectbox("Nivel de detalle", options=list(DETAIL_LEVELS),
                                    format_func=DETAIL_LEVELS.get, ...)
    with c3:
        output_format = st.selectbox("Formato de salida", options=list(OUTPUT_FORMATS),
                                     format_func=OUTPUT_FORMATS.get, ...)
    enviar = st.form_submit_button("Estimar", type="primary")

# Resultado de la elección:
# project_type == "web_saas"      (el usuario vio "SaaS web")
# detail_level == "detailed"     (vio "Detallado")
# output_format == "phases_table" (vio "Tabla de fases")
```

Paso 2 · El payload viaja como JSON (streamlit_app.py)

`_armar_payload()` está fuera de `main()` para que los tests usen el mismo armado que la interfaz: una clave mal escrita rompería el contrato sin ningún error local.

```
def _armar_payload(descripcion, project_type, detail_level, output_format) -> dict:
    return {
        "description": descripcion,
        "project_type": project_type,
        "detail_level": detail_level,
        "output_format": output_format,
    }

limpio = descripcion.strip()
error = _error_de_longitud(limpio)          # 20-2000 caracteres, antes de
llamar a la API
payload = _armar_payload(limpio, project_type, detail_level, output_format)

# En _estimar():
respuesta = cliente.post(
    f"{api_base}/api/v1/estimate",
    json=payload,
    params={"prompt_version": prompt_version} if prompt_version else None,
    timeout=None,
)
```

Lo que llega a la API:

```
POST /api/v1/estimate?prompt_version=v2
Content-Type: application/json

{
  "description": "Plataforma SaaS de reservas para clínicas: agenda online,
pagos con tarjeta, recordatorios por e-mail y panel de administración.",
  "project_type": "web_saas",
  "detail_level": "detailed",
  "output_format": "phases_table"
}
```

Paso 3 · FastAPI lo convierte en EstimationRequest (app/schemas/estimation.py)

```
class ProjectType(str, Enum):
    MOBILE_APP = "mobile_app"
    WEB_SAAS = "web_saas"
    INTERNAL_TOOL = "internal_tool"
    DATA_PIPELINE = "data_pipeline"

class DetailLevel(str, Enum):
    SUMMARY = "summary"
    MEDIUM = "medium"
    DETAILED = "detailed"

class OutputFormat(str, Enum):
    PHASES_TABLE = "phases_table"
    LINE_ITEMS = "line_items"
    NARRATIVE = "narrative"

class EstimationRequest(BaseModel):
    description: str = Field(min_length=20, max_length=2000)
    project_type: ProjectType
    detail_level: DetailLevel
    output_format: OutputFormat
    # + _validar_techo_del_operador(): techo configurable de Settings
```

```
@router.post("/estimate", response_model=EstimationResponse, ...)
async def estimate(
    request: EstimationRequest,          # ← FastAPI convierte el
    JSON acá
    prompt_version: str | None = Query(default=None, ...),
    llm_wrapper=Depends(get_llm_wrapper),
) -> EstimationResponse:
    version = _resolver_version(prompt_version)      # "v2"
    system_prompt, user_prompt = render_estimation_prompt(request, version)
    ...
```

Objeto resultante (salida real de `repr()`):

```
EstimationRequest(
  description='Plataforma SaaS de reservas para clínicas: agenda online, ...',
  project_type=<ProjectType.WEB_SAAS: 'web_saas'>,
  detail_level=<DetailLevel.DETAILED: 'detailed'>,
  output_format=<OutputFormat.PHASES_TABLE: 'phases_table'>,
)
```

Los campos ya no son str: son miembros de enum. Un valor fuera del contrato se rechaza con **422** antes de renderizar el prompt o llamar al LLM. Por ejemplo, "project_type": "web_app" devuelve:

```
{"type": "enum", "loc": ["body", "project_type"],
 "msg": "Input should be 'mobile_app', 'web_saas', 'internal_tool' or 'data_pipeline'",
 "input": "web_app"}
```

Paso 4 · Del objeto tipado al prompt (app/prompts/loader.py y v2/system.j2)

El loader pasa a Jinja2 el `.value` de cada enum, y la plantilla elige las instrucciones con esos valores.

```
# app/prompts/loader.py · render_estimation_prompt()
context = {
    "description": request.description,
    "project_type": request.project_type.value,      # "web_saas"
    "detail_level": request.detail_level.value,      # "detailed"
    "output_format": request.output_format.value,    # "phases_table"
    "rates": ..., "examples": ..., "version": version,
}
system = _env.get_template(f"estimation/{version}/system.j2").render(**context)
user = _env.get_template(f"estimation/{version}/user.j2").render(**context)
```

```
{# app/prompts/estimation/v2/system.j2 #}
{% if output_format == "phases_table" %} ... {% elif output_format ==
"line_items" %} ...
{% if detail_level == "summary" %} ... {% elif detail_level == "detailed" %} ...
```

Prompt de usuario renderizado (user.j2):

```
<project_description>
Plataforma SaaS de reservas para clínicas: agenda online, pagos con tarjeta,
recordatorios
por e-mail y panel de administración.
</project_description>

Tipo de proyecto: web_saas.
Estima este proyecto siguiendo las reglas anteriores.
```

Fragmentos del prompt de sistema renderizado para esta elección (salida real):

```
<output_format>
El usuario pidió output_format = "phases_table". Presenta la estimación así:
Una tabla Markdown con las columnas: Fase | Semanas | Coste (EUR) | Confianza
(%).
Después de la tabla, una línea "Totales" con las horas totales, el coste total
en EUR y la
duración total en semanas, y una línea "Equipo" con la composición del equipo.
El valor phases_table identifica este formato.
</output_format>

<detail_level>
El usuario pidió detail_level = "detailed":
Para cada fase, enumera los supuestos en viñetas (límites del alcance,
dependencias,
riesgos). Señala al menos tres riesgos explícitos, cada uno con su mitigación.
</detail_level>
```

Conclusión

- Las etiquetas en castellano existen solo en la interfaz; el contrato entre el frontend y la API son las claves de los enums.
- Pydantic valida en el borde: todo lo que llega a `render_estimation_prompt()` ya es una de las $4 \times 3 \times 3 = 36$ combinaciones posibles.